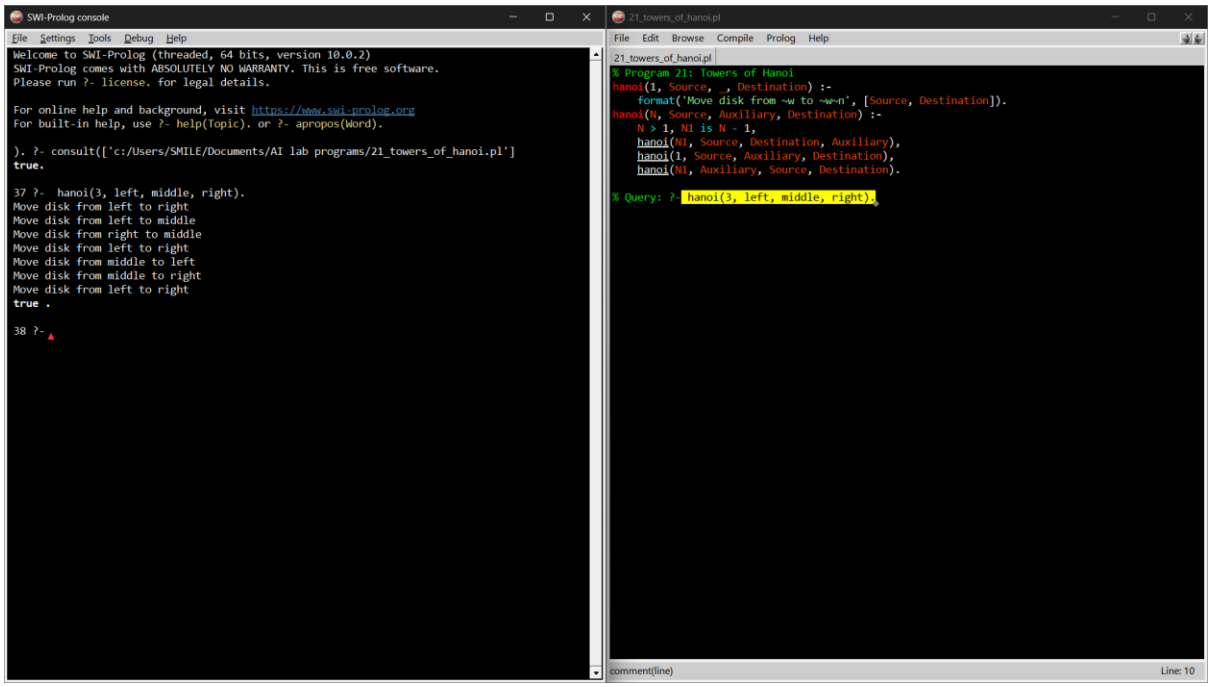
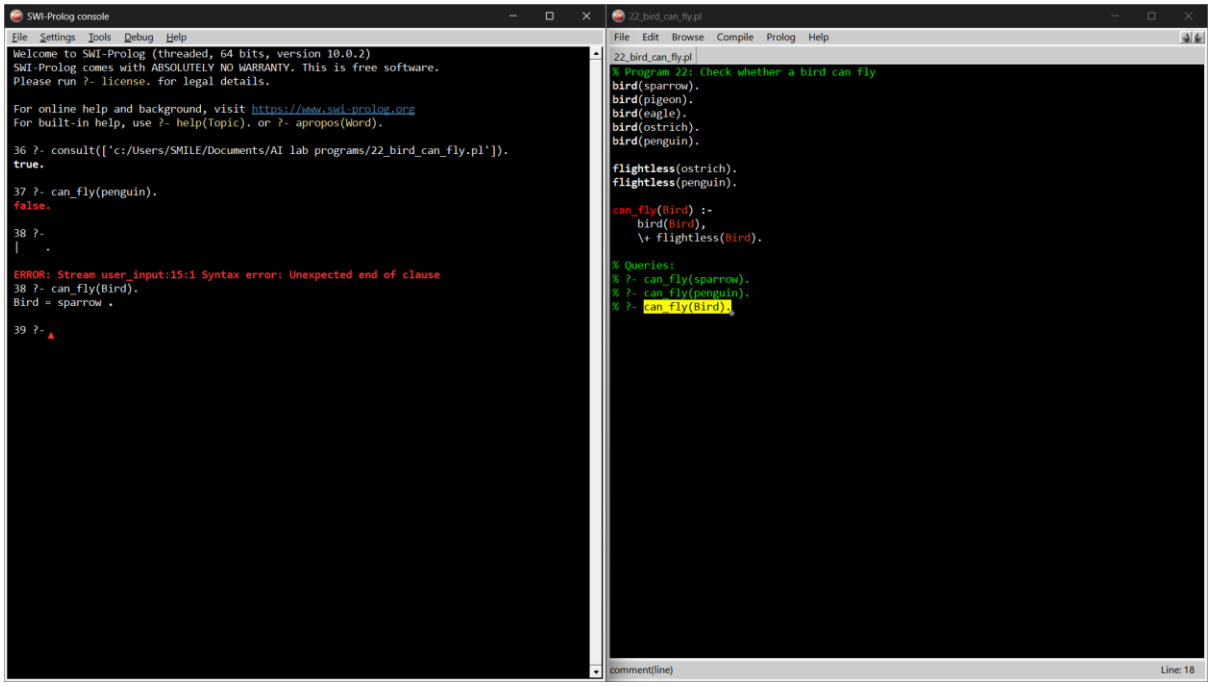
Program 21-25

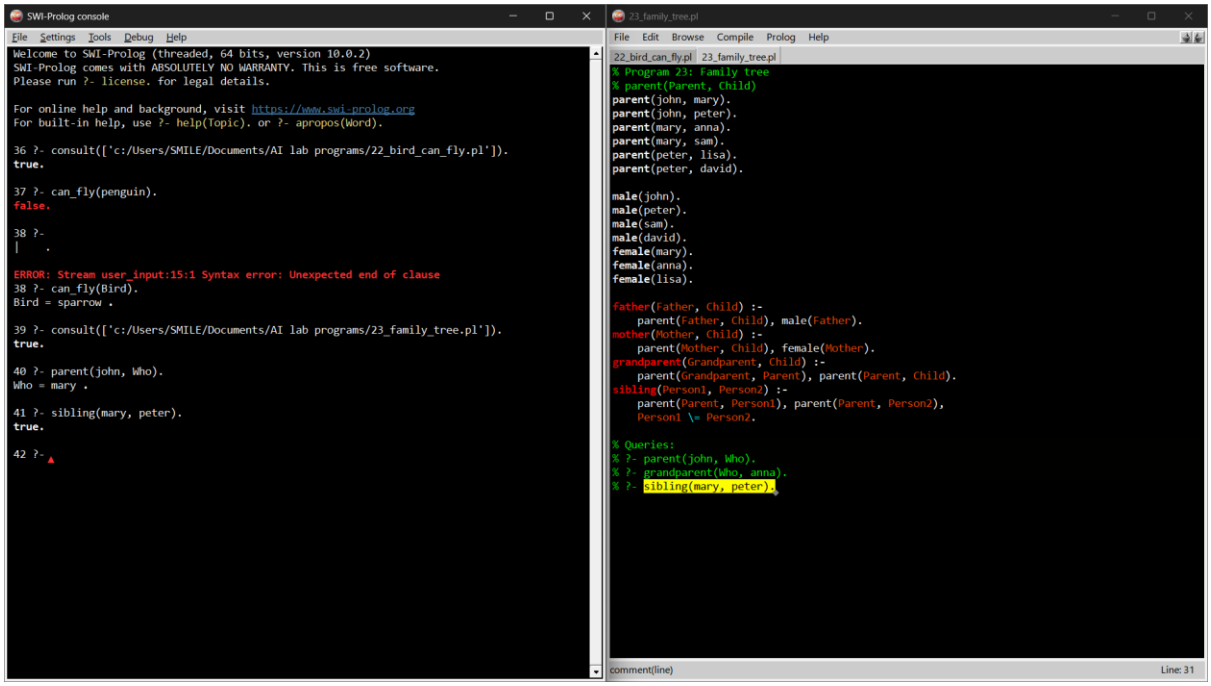
Program 21:



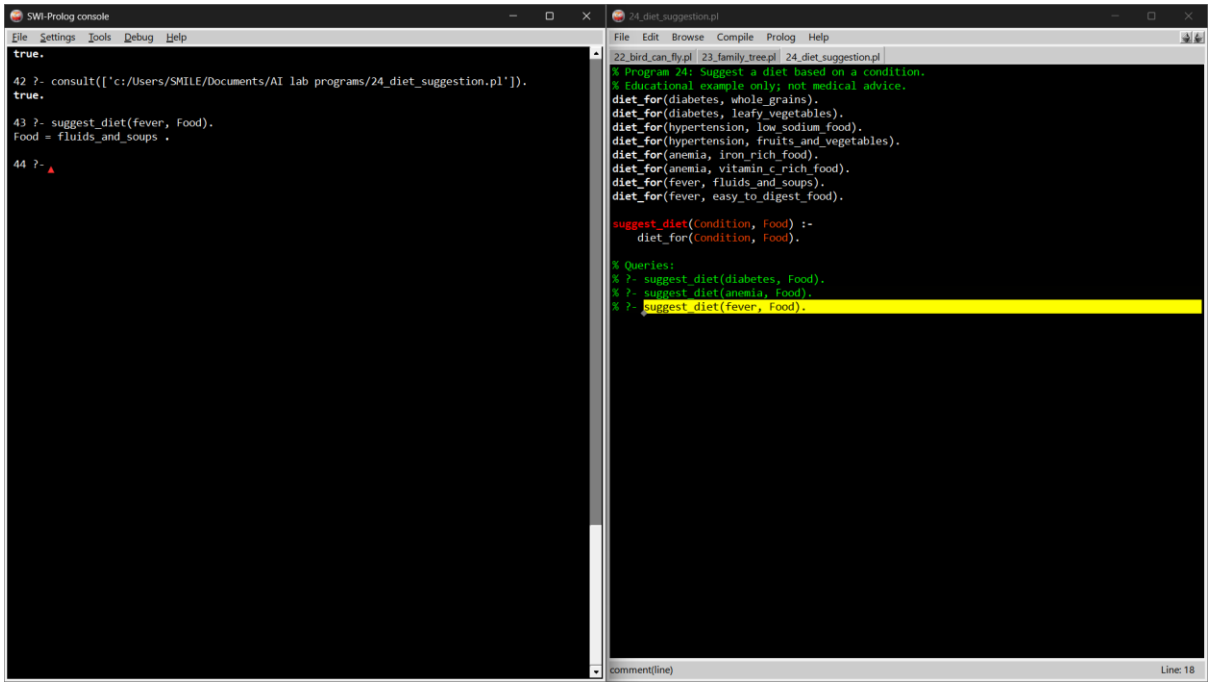
Program 22:


SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
36 ?- consult(['c:/Users/SMILE/Documents/AI lab programs/22_bird_can_fly.pl']).
true.
37 ?- can_fly(penguin).
false.
38 ?-
| .
ERROR: Stream user_input:15:1 Syntax error: Unexpected end of clause
38 ?- can_fly(Bird).
Bird = sparrow .
39 ?-
22_bird_can_fly.pl
File Edit Browse Compile Prolog Help
22_bird_can_fly.pl
% Program 22: Check whether a bird can fly
bird(sparrow).
bird(pigeon).
bird(eagle).
bird(ostrich).
bird(penguin).
flightless(ostrich).
flightless(penguin).
can_fly(Bird) :-
bird(Bird),
\+ flightless(Bird).
% Queries:
% ?- can_fly(sparrow).
% ?- can_fly(penguin).
% ?- can_fly(Bird).
comment(line)
Line: 18

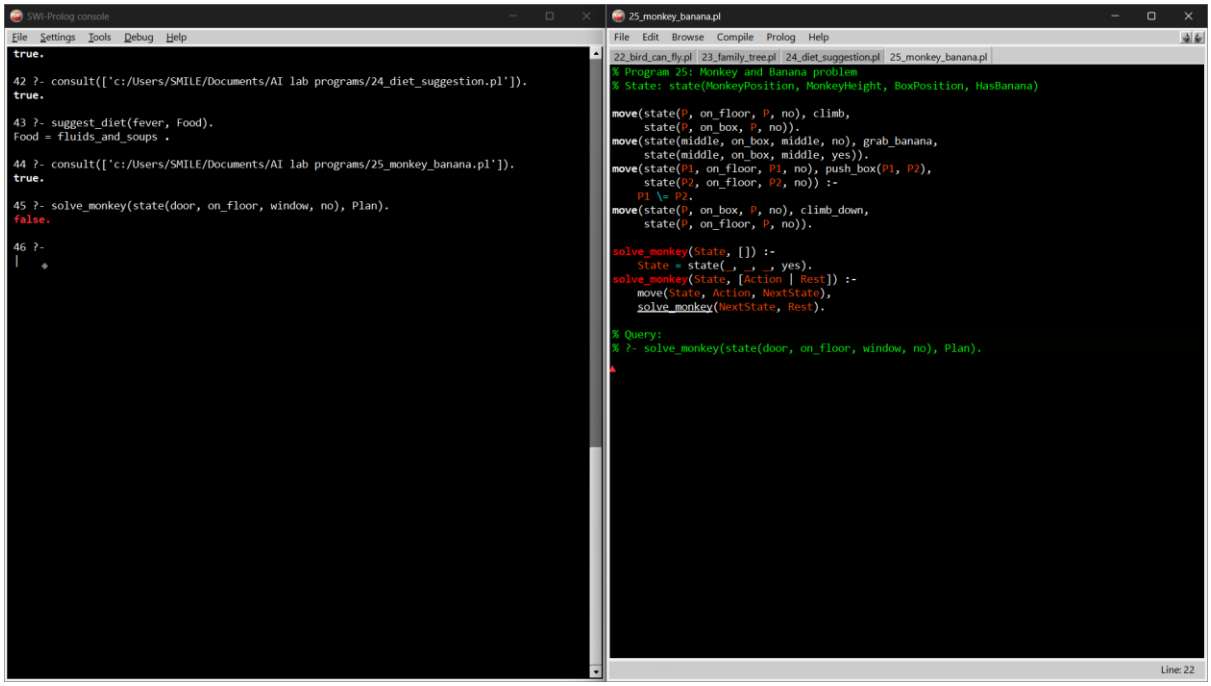
Program 23:


SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
36 ?- consult(['c:/Users/SMILE/Documents/AI lab programs/22_bird_can_fly.pl']).
true.
37 ?- can_fly(penguin).
false.
38 ?-
| .
ERROR: Stream user_input:15:1 Syntax error: Unexpected end of clause
38 ?- can_fly(Bird).
Bird = sparrow .
39 ?- consult(['c:/Users/SMILE/Documents/AI lab programs/23_family_tree.pl']).
true.
40 ?- parent(john, Who).
Who = mary .
41 ?- sibling(mary, peter).
true.
42 ?-
23_family_tree.pl
File Edit Browse Compile Prolog Help
22_bird_can_fly.pl 23_family_tree.pl
% Program 23: Family tree
% parent(Parent, Child)
parent(john, mary).
parent(john, peter).
parent(mary, anna).
parent(mary, sam).
parent(peter, lisa).
parent(peter, david).
male(john).
male(peter).
male(sam).
male(david).
female(mary).
female(anna).
female(lisa).
father(Father, Child) :-
parent(Father, Child), male(Father).
mother(Mother, Child) :-
parent(Mother, Child), female(Mother).
grandparent(Grandparent, Child) :-
parent(Grandparent, Parent), parent(Parent, Child).
sibling(Person1, Person2) :-
parent(Parent, Person1), parent(Parent, Person2),
Person1 \= Person2.
% Queries:
% ?- parent(john, Who).
% ?- grandparent(Who, anna).
% ?- sibling(mary, peter).
comment(line)
Line: 31

Program 24:


SWI-Prolog console
File Settings Tools Debug Help
true.
42 ?- consult(['c:/Users/SMILE/Documents/AI lab programs/24_diet_suggestion.pl']).
true.
43 ?- suggest_diet(fever, Food).
Food = fluids_and_soups .
44 ?-
24_diet_suggestion.pl
File Edit Browse Compile Prolog Help
22_bird_can_fly.pl 23_family_tree.pl 24_diet_suggestion.pl
% Program 24: Suggest a diet based on a condition.
% Educational example only; not medical advice.
diet_for(diabetes, whole_grains).
diet_for(diabetes, leafy_vegetables).
diet_for(hypertension, low_sodium_food).
diet_for(hypertension, fruits_and_vegetables).
diet_for(anemia, iron_rich_food).
diet_for(anemia, vitamin_c_rich_food).
diet_for(fever, fluids_and_soups).
diet_for(fever, easy_to_digest_food).
suggest_diet(Condition, Food) :-
diet_for(Condition, Food).
% Queries:
% ?- suggest_diet(diabetes, Food).
% ?- suggest_diet(anemia, Food).
% ?- suggest_diet(fever, Food).
comment(line)
Line: 18

Program 25:


SWI-Prolog console
File Settings Tools Debug Help
true.
42 ?- consult(['c:/Users/SMILE/Documents/AI lab programs/24_diet_suggestion.pl']).
true.
43 ?- suggest_diet(fever, Food).
Food = fluids_and_soups .
44 ?- consult(['c:/Users/SMILE/Documents/AI lab programs/25_monkey_banana.pl']).
true.
45 ?- solve_monkey(state(door, on_floor, window, no), Plan).
false.
46 ?-
25_monkey_banana.pl
File Edit Browse Compile Prolog Help
22_bird_can_fly.pl 23_family_tree.pl 24_diet_suggestion.pl 25_monkey_banana.pl
% Program 25: Monkey and Banana problem
% State: state(MonkeyPosition, MonkeyHeight, BoxPosition, HasBanana)
move(state(P, on_floor, P, no), climb,
state(P, on_box, P, no)).
move(state(middle, on_box, middle, no), grab_banana,
state(middle, on_box, middle, yes)).
move(state(P1, on_floor, P1, no), push_box(P1, P2),
state(P2, on_floor, P2, no)) :-
P1 \= P2.
move(state(P, on_box, P, no), climb_down,
state(P, on_floor, P, no)).
solve_monkey(State, []) :-
State = state(_, _, _, yes).
solve_monkey(State, [Action | Rest]) :-
move(State, Action, NextState),
solve_monkey(NextState, Rest).
% Query:
% ?- solve_monkey(state(door, on_floor, window, no), Plan).
Line: 22